LoanMate AI — Viva Cheat Sheet

Generative AI + Retrieval-Augmented Generation (RAG) Based Loan Assistant · Condensed Presentation & Viva Prep

60-Second Explanation

🎤 **SAY THIS**

"LoanMate AI is a voice-first loan assistant — customers speak in English, Hindi, or Hinglish and Gemini Live has a live conversation with them. The model is agentic: for account facts (EMI, balance) it calls a deterministic tool that reads real customer data, so numbers are never invented. For general policy/product questions ('what documents do I need', 'what's your prepayment policy'), there's no single customer record to check — so I built a RAG pipeline: 11 Markdown knowledge documents are split into 75 semantic chunks, embedded with a Gemini embedding model, and stored in a local FAISS vector index. When the agent decides a question is a knowledge question, it calls a tool (`search_loan_knowledge`) that embeds the query, searches the index, and — only if results score above a 0.65 relevance threshold — hands the real matched text back so the model answers using our actual documents instead of guessing."

Problem → Solution

Problem: loan/EMI info is simple in content but tedious to find (login → dashboard → hunt for the right widget), and a bare LLM asked about it would just guess. **Solution:** a spoken conversation with an agentic model that pulls real account data via tools and real policy text via RAG — never guessing either.

Architecture — Two Flows

Online (live call)

```
User speaks → Next.js Frontend
→ LiveKit Cloud (WebRTC)
→ Realtime Agent worker
→ Gemini Live (gemini-3.8-live)
→ decides: tool / RAG / direct?
  if RAG → search_loan_knowledge
  → embed_query() → FAISS search
  → chunks ≥0.65 returned
→ Gemini composes grounded reply
→ back through LiveKit → User
```

Offline (one-time indexing)

```
backend/knowledge/*.md (11 files)
→ document_loader.py (load)
→ chunker.py (75 chunks,
   split on "## " headings)
→ embeddings.py
   (gemini-embedding-2, 768-dim)
→ vector_store.py
   (FAISS IndexFlatIP)
→ saved: loanmate.index +
   metadata.json
```

⚠️ **KEY POINT**

RAG is a **tool** the agent chooses to call for general knowledge questions only — customer facts/calcs use 9 other deterministic tools. The app never rebuilds the index at runtime; only `ingest.py` does.

RAG Pipeline — File by File

File	Job	Key fact
`document_loader.py`	Loads all .md files	Tags each with document_type (folder) + topic (filename)
`chunker.py`	Splits into semantic chunks	Splits on `##` headings, not blind chars → 75 chunks, no overlap
`embeddings.py`	Text → vector	`gemini-embedding-2` , 768-dim (truncated from 3072); RETRIEVAL_DOCUMENT vs RETRIEVAL_QUERY task types
`vector_store.py`	Stores & searches vectors	Local FAISS `IndexFlatIP` (exact, not ANN) — cosine via normalized inner product
`retriever.py`	Embeds query, searches, filters	top_k=3; `MIN_RELEVANCE_SCORE=0.65` (empirical: on-topic 0.71–0.86, off-topic 0.53–0.59)
`rag_service.py`	Agent-facing orchestrator	Formats source-attributed text; returns explicit "not found" message if nothing clears threshold
`ingest.py`	Offline index builder	Run manually: `python -m backend.rag.ingest`

Where Is Generative AI? (most-asked question)

🎤 **SAY THIS**

Retrieval (FAISS) is **search**, not generation — it only finds existing text. **Generation** is Gemini Live composing a brand-new sentence from that retrieved text or a tool's returned number. Every spoken reply is dynamically generated, never a fixed lookup table.

Key Distinctions (viva favorites)

Question	Answer
RAG vs Fine-tuning?	RAG = retrieve at query time, no training, instant updates via file edit + re-index. Fine-tuning = retrain model weights, slow/costly. We use RAG — no model was fine-tuned.
RAG vs plain database?	DB matches exact fields (`WHERE category=X`); vector search matches by *meaning*, so differently-worded or Hinglish questions still match the right section.
Why not just ask Gemini directly?	Gemini has never seen our fictional bank's specific policies — it would guess. RAG grounds the answer in our real, editable, traceable documents.
Does RAG eliminate hallucination?	**No.** It reduces the risk (real text + 0.65 threshold + explicit "not found" message) but doesn't guarantee correctness.
Is RAG itself Generative AI?	No — it's a technique combining retrieval (search) with generation (LLM). Retrieval alone produces no new text.
Are embeddings from the same model as the chatbot?	No — `gemini-embedding-2` (vectors only) is separate from `gemini-3.8-live` (the conversational model).

Trap Questions — Quick Answers

🚨 **Does vector search generate the answer?**
No — it finds chunks; Gemini Live writes the answer.

🚨 **Does RAG train the LLM?**
No — retrieval happens at inference time; model weights never change.

🚨 **Is RAG used on every question?**
No — only when the agent decides it's a general knowledge question, not customer-specific.

🚨 **Is a vector DB just a normal DB?**
No — it does similarity search over vectors, not exact-match queries.

🚨 **Why exact FAISS index, not HNSW/ANN?**
Only 75 chunks — exact search is already sub-millisecond; ANN would trade accuracy for no real benefit here.

🚨 **What if nothing relevant is found?**
Score stays below 0.65 → explicit "no relevant info found" message, not a guess.

30-Second Answers

What is RAG?
Retrieval-Augmented Generation — retrieve real matching text before the LLM answers, so it's grounded, not guessed.

What is an embedding?
A 768-number vector representing a text's meaning; similar meanings → nearby vectors.

What is a vector store?
A database (local FAISS here) built to find the most similar stored vectors to a query vector.

What does the retriever do?
Embeds the query, searches FAISS for top-3, discards anything below 0.65 similarity.

What is top-k?
Return only the k best matches (k=3 here) instead of the whole knowledge base.

Why chunk documents?
So each embedding is topically focused — improves match precision vs. embedding a whole file at once.

Terminology — One-Line Definitions

LLM — pretrained model that understands/generates language.

GenAI — AI that creates new content, not fixed lookups.

RAG — retrieve real text, then generate a grounded answer.

Embedding — numeric vector representing text meaning.

Vector — ordered list of numbers (768 here).

Vector store — DB optimized for similarity search (FAISS).

Chunk / Chunking — a focused piece of a document / the act of splitting into pieces.

Semantic search — finding text by meaning, not keywords.

Grounding — tying an answer to real, checkable source text.

Hallucination — LLM confidently stating something false.

Top-K — only the K best-scoring results returned.

Fine-tuning — retraining model weights (not used here).

Metadata — source_file/topic/section tags on each chunk.

Reranking / Hybrid search — future-work techniques, not implemented.

Inference vs Training — running a trained model (all of LoanMate) vs. updating its weights (never done here).

Tech Stack (verified only)

Layer	Tech
Frontend	Next.js 16 / React 19 (primary); Streamlit (legacy)
Backend	FastAPI, Python 3.13
Realtime	LiveKit Cloud + Agents (WebRTC)

Layer	Tech
Conversation LLM	Gemini Live — `gemini-3.8-live`
Embeddings	`gemini-embedding-2` (768-dim)
Vector index	FAISS `IndexFlatIP` (local, exact)

Limitations & Future Scope (say honestly)

Limitations

- Small demo KB (11 docs / 75 chunks)
- RAG cold-start latency: 4.8–8.6s first call, ~568ms warm
- RAG doesn't guarantee correctness, only reduces risk
- Mock customer data, no real telephony/auth

Future Scope

- ANN index (HNSW) if KB grows large
- Reranking, hybrid search, metadata filtering
- Source citations shown to user; real auth
- Formal precision/recall evaluation framework

Demo Script — Questions to Ask Live

Question	Demonstrates
"What is my EMI?"	Deterministic tool (`get_emi`) — customer data, no RAG
"What documents do I need for a personal loan?"	RAG → `required_documents.md`
"Prepayment karne pe koi charges hain kya?" (Hinglish)	RAG working across languages
"Agar main do lakh prepay karun, aur charges?"	Hybrid — calculator tool + RAG together
"What's the weather today?"	Off-topic → agent honestly declines (threshold at work)

> 🎤 **SAY THIS after any RAG answer**
> "This answer isn't from the model's general knowledge — it first retrieved relevant text from our own knowledge base and then generated this explanation grounded in that text."

Mini Mock Viva

T: "Where is Generative AI in your project?"
S: "Gemini Live — it generates every reply dynamically and decides which tool to call."

T: "Isn't retrieval just search?"
S: "Yes — retrieval only finds chunks; Gemini Live is what generates the actual answer."

T: "Why not fine-tune Gemini on your documents instead?"
S: "Fine-tuning needs retraining every time a policy changes. RAG just needs a file edit and a re-index, and every answer is traceable to a source chunk."

T: "Where did 0.65 come from — did you just guess it?"
S: "No — measured scores: on-topic 0.71–0.86, off-topic 0.53–0.59; 0.65 sits in that empirical gap."

T: "Does RAG guarantee correct answers?"
S: "No — it reduces hallucination risk by grounding in real text, but doesn't eliminate it."

T: "If I gave you 10,000 more documents tomorrow, what would break?"
S: "The exact FAISS search would get slower linearly with size — I'd switch to an approximate index like HNSW at that scale."

If You Remember Only 5 Things

1. **Retrieval ≠ Generation:** FAISS finds chunks; Gemini Live writes the answer.
2. RAG fires only for **general knowledge questions** — customer facts use 9 separate deterministic tools.
3. Numbers to know: **11 docs → 75 chunks → 768-dim → top_k=3 → threshold 0.65.**
4. Indexing is **offline/one-time** (`ingest.py`); querying is **live**, only when RAG is triggered.
5. **RAG reduces hallucination risk — it does not eliminate it.** Never claim 100% accuracy.

LoanMate AI uses a RAG pipeline to retrieve relevant information from a structured loan knowledge base and a generative AI model to produce a natural-language answer grounded in that context. Good luck.